

EchoEats

Dynamic Pricing Food Waste Marketplace

Assignment 2 — Architecture Selection, Detailed Design & Ethical Planning

SWEN 360 / CMPE 460 · Sprint 2 · Spring 2026

Course	SWEN 360 / CMPE 460
Assignment	Assignment 2
Sprint	Sprint 2 — 4 Weeks
Semester	Spring 2026
Scrum Master	Rashid Bomtia

Team Members

Name	Role
Yousif Shakeeb	Developer
Mohamed Elshamy	Developer
Rashid Bomtia	Scrum Master / Developer
Ali Al Buainain	Developer
Hussain Yusuf	Developer

PART 1 — TEAM SECTION (Identical in All Submissions)

A. Team Deliverables

A.1 — Agile Process & Project Management

Sprint 2 Kanban Board Progression

Sprint Start — Week 5

BACKLOG	TO DO	IN PROGRESS	DONE
0 issues	8 issues	0 issues	0 issues
—	[Design] Architecture Research Rashid, Yousif · 3 SP	—	—
—	[Design] Decision Matrix Scoring Full Team · 2 SP	—	—
—	[Feature] UML Class Diagram — Full Domain Model Yousif · 5 SP	—	—
—	[Feature] Sequence Diagram Mohamed · 5 SP	—	—
—	[Feature] Component Diagram Ali · 5 SP	—	—
—	[Feature] Ethical Statement Hussain · 3 SP	—	—
—	[Design] Wireframe Polish Yousif, Ali · 3 SP	—	—
—	[Feature] ML Pricing Optimisation Rashid, Hussain · 4 SP	—	—

Mid-Sprint — Week 6

BACKLOG	TO DO	IN PROGRESS	DONE
0 issues	4 issues	2 issues	2 issues
—	[Feature] Component Diagram Ali · 5 SP	[Feature] UML Class Diagram — Full Domain Model Yousif · 5 SP	[Design] Architecture Research Rashid, Yousif · 3 SP
—	[Feature] Ethical Statement Hussain · 3 SP	[Feature] Sequence Diagram Mohamed · 5 SP	[Design] Decision Matrix Scoring Full Team · 2 SP
—	[Design] Wireframe Polish Yousif, Ali · 3 SP	—	—
—	[Feature] ML Pricing Optimisation Rashid, Hussain · 4 SP	—	—

Mid-Sprint — Week 7

BACKLOG	TO DO	IN PROGRESS	DONE
0 issues	0 issue	3 issues	4 issues (15 SP)
—	—	[Feature] Component Diagram Ali · 5 SP	[Design] Architecture Research Rashid, Yousif · 3 SP
—	—	[Feature] Ethical Statement Hussain · 3 SP	[Design] Decision Matrix Scoring Full Team · 2 SP
—	—	[Design] Wireframe Polish Yousif · 3 SP	[Feature] UML Class Diagram — Full Domain Model Yousif · 5 SP
—	—	—	[Feature] Sequence Diagram Mohamed · 5 SP

Sprint End — Week 8

BACKLOG	TO DO	IN PROGRESS	DONE ✓
0 issues	1 issue (4 SP — Deferred)	0 issues	7 issues (22 SP)
—	[Feature] ML Pricing Optimisation Rashid, Hussain · 4 SP → <i>Deferred</i>	—	[Design] Architecture Research Rashid, Yousif · 3 SP

—	—	—	[Design] Decision Matrix Scoring Full Team · 2 SP
—	—	—	[Feature] UML Class Diagram — Full Domain Model Yousif · 5 SP
—	—	—	[Feature] Sequence Diagram Mohamed · 5 SP
—	—	—	[Feature] Component Diagram Ali · 5 SP
—	—	—	[Feature] Ethical Statement Hussain · 3 SP
—	—	—	[Design] Wireframe Polish Yousif, Ali · 3 SP

Sprint Planning

Field	Detail
Date	Week 5 — Sprint 2 Kickoff
Duration	90 minutes
Attendees	All 5 members (Rashid Bomtia — Scrum Master)
Sprint Goal	Architecture Selection, Detailed Design, Ethical Planning, and all UMLs
Scrum Master	Rashid Bomtia (rotated from Yousif Shakeeb in Sprint 1)
Product Owner	Instructor
Definition of Done	All diagrams have been reviewed by not less than 2 team members, and the ethical statement has been approved by the team, document merged into GitHub main branch.

Daily Stand-up Minutes — 8 Stand-ups

Stand-up #1 · Week 5 — Mon (Mar 10)	
<i>Sprint 2 Kickoff: Architecture research begins</i>	
Mohamed Elshamy	Go over items from Sprint retrospective. Prepared to make contribution to matrix.
Rashid Bomtia	A study was started to evaluate the layered architecture and microservices architecture style. I will prepare my notes first by Wednesday.
Ali Al Buainain	Examined wireframes for Sprint 1. Prepared to back component diagram in Week 6.
Hussain Yusuf	Checked Sprint 1 retro. The ethical statement research will be conducted after architectural decisions.
Yousif Shakeeb	Research on architecture jointly with Rashid. Going over Sprint 1 handover documents for new Scrum Master Rashid.
Blockers	None.

Stand-up #2 · Week 5 — Wed (Mar 12)	
<i>Architecture research complete — decision matrix started</i>	
Mohamed Elshamy	Research on architecture done. Filling in scores for Microservices category in decision matrix.
Rashid Bomtia	Research Inspired by Architecture. The entire team had a meeting today to offer help on decision matrix.
Ali Al Buainain	Helped with client-server analysis research. Establishing the scoring criteria with the team now.
Hussain Yusuf	Helped compare different architectures. Checked security and team fit criterion.
Yousif Shakeeb	Research on structure completed. Writing the comparison story for the report.
Blockers	None.

Stand-up #3 · Week 6 — Sat (Mar 15)	
<i>Decision matrix drafted — class diagram work begins</i>	
Mohamed Elshamy	Decision Matrix Finished. Commencing review of relationships of class diagram entities
Rashid Bomtia	The decision matrix was successfully completed. Wireframe designs for Listing Detail and Vendor Dashboard.
Ali Al Buainain	Matrix drafter review Next week will begin component diagram research.
Hussain Yusuf	Approved matrix of decisions. Initiating background reading on GDPR and ethical design.
Yousif Shakeeb	Starting UML class diagram. Discussed user-consumer-vendor hierarchy and subclass structure today.
Blockers	The schema is unclear as some are debating a potential consumer vs vendor attribute split.

Stand-up #4 · Week 6 — Mon (Mar 17)	
<i>Architecture analysis report first draft complete</i>	
Mohamed Elshamy	The first draft of the architecture report has been completed. Initial sequence diagram for reserving a listing.
Rashid Bomtia	Documenting a Decision on Minimising Location Data.
Ali Al Buainain	Examining the design report. Prepare to begin a component diagram this week.
Hussain Yusuf	Analysis of class diagram for ethical concerns (user data).
Yousif Shakeeb	Hierarchy of the class diagram is finalised. The composition of ImpactProfile was added afterwards. going through gamification user stories.
Blockers	None.

Stand-up #5 · Week 7 — Wed (Mar 19)	
<i>Sequence diagram complete — component diagram starts</i>	
Mohamed Elshamy	The sequence diagram for reserving a listing consisting of 15 steps has been completed and submitted for PR review.
Rashid Bomtia	The initial draft of the Listing Detail wireframe is done. The decision log of ADR-03 has been prepared. The GPS is not to stay with the vendor for more than 48 hours. The vendor will be given the name of the area.
Ali Al Buainain	Starting component diagram. Describing a five-stage configuration and layer-specific connections.
Hussain Yusuf	Examined Sequence Diagram First draft of the opening ethical statement.
Yousif Shakeeb	I looked over the sequence diagram PR and noticed that the JWT validation step was missing. It will be fixed by the time it is merged. Initiating adjustments on the wireframe.
Blockers	The Pricing Optimisation will be intentionally delayed due to error encountering.

Stand-up #6 · Week 7 — Mon (Mar 24)	
<i>Component diagram drafted — ethical statement in progress</i>	
Mohamed Elshamy	Analyzing component scheme. I will support Hussain if they need help.
Rashid Bomtia	After review, the PR sequence diagram is approved. While I continue my review of the draft component diagram, the discussion on scope of ML pricing optimization has been deferred and descoped from Sprint 2. Blockage alter log.
Ali Al Buainain	The initial version of the component diagram is completed. Team review awaited.
Hussain Yusuf	The first draft of the ethical statement is complete. It covers issues of privacy, fairness, and safety.
Yousif Shakeeb	Refined wireframe for screens 1-2 Creating dashboards and confirmation screens for vendors.
Blockers	None.

Stand-up #7 · Week 8 — Wed (Mar 26)	
<i>Ethical statement drafted — full team diagram review</i>	
Mohamed Elshamy	Complete diagram review ongoing. I noticed an error in the sequence diagram step numbers. Will fix it soon.
Rashid Bomtia	The wireframe for the vendor's dashboard has been consolidated. Core does ADR-03. Perusing all illustrations.
Ali Al Buainain	After receiving feedback from the team, the interface labels weren't present on the layer between layers.
Hussain Yusuf	Team approves the ethical statement. A section on privacy trade-off was added.
Yousif Shakeeb	The wireframes for four screens have completed. Diagrams to be Reviewed for Compatibility with Presentation Layer Specification.
Blockers	In the same session, addressed minor inconsistency issues in the diagrams such as the class diagram legend and sequence step numbering.

Stand-up #8 · Week 8 — Mon (Mar 31)	
<i>All diagrams peer-reviewed — final document assembly</i>	
Mohamed Elshamy	All diagrams approved by peers. Compiling each part of the report.
Rashid Bomtia	I integrated all the diagrams into the main one. Begins finalizing and blocking documents and logs.
Ali Al Buainain	The final modified version component diagram has been merged. Composing personal contribution section.
Hussain Yusuf	Ethical statement integrated. Contributory sections for individual.
Yousif Shakeeb	All diagrams and wireframes merged. Validation and review of a evidence and reliable contribution section in writing paper via programming.
Blockers	None. The sprint will close on Friday of week 8.

Sprint Review

The Sprint Review was a successful event of 45 minutes in the 8th week. The team presented UML diagrams, showing all the drawings, screen flow and so on. We were allowed to take an N-tier solution. Before submission, the meaning of the Observer pattern was clarified.

Sprint Retrospective

Begin research on the architecture before future sprints. If diagram integration takes place on the last day, a merge conflict may arise. Stand up meetings are returning to the parking lot. The parties reached an agreement to meet synchronously and add impediment log. Rashid will face the obstruction log. As of Sprint 3, add ethics review to standing agenda.

GitHub Repository

Repository: Repository: <https://github.com/YMShakeeb/echoeats-web>

Each of the five members developed their features in feature branches (name: feature/<name>) and made contributions. All pull requests to develop must receive two peer reviews prior to being merged into main at sprint close. This detected a missing JWT validation step and a layer interface mismatch, at a point before main.

Member	Commits	Key Contribution
Rashid Bomtia	12	Burndown chart, ADR-03, impediment log.
Yousif Shakeeb	10	Authored full UML Class Diagram (all 9 entities), defined Presentation Layer in Component Diagram, reviewed PRs, and wrote Sprint 1 handover documentation.
Mohamed Elshamy	9	API Gateway design for reserve flow sequence diagram.
Ali Al Buainain	8	Interface specifications of layer components.
Hussain Yusuf	7	Ethical Design Statement, an outline of the data-layer schema.

Burn-down Chart — Sprint 2

During Sprint 2, we promised to deliver 26 story points. We ended up delivering 22 story points, which is 85% velocity. Following discussions on the scope during Stand-up #5, the team purposely opted not to take up the remaining other 4 points (ML pricing optimisation) for the Sprint. The burndown closely followed the ideal line during Weeks 5-6. There was a little plateau at Week 7 when the ethics review start to take place. And there followed a recovery at Week 8.



A.2a — Architecture Analysis Report (B1)

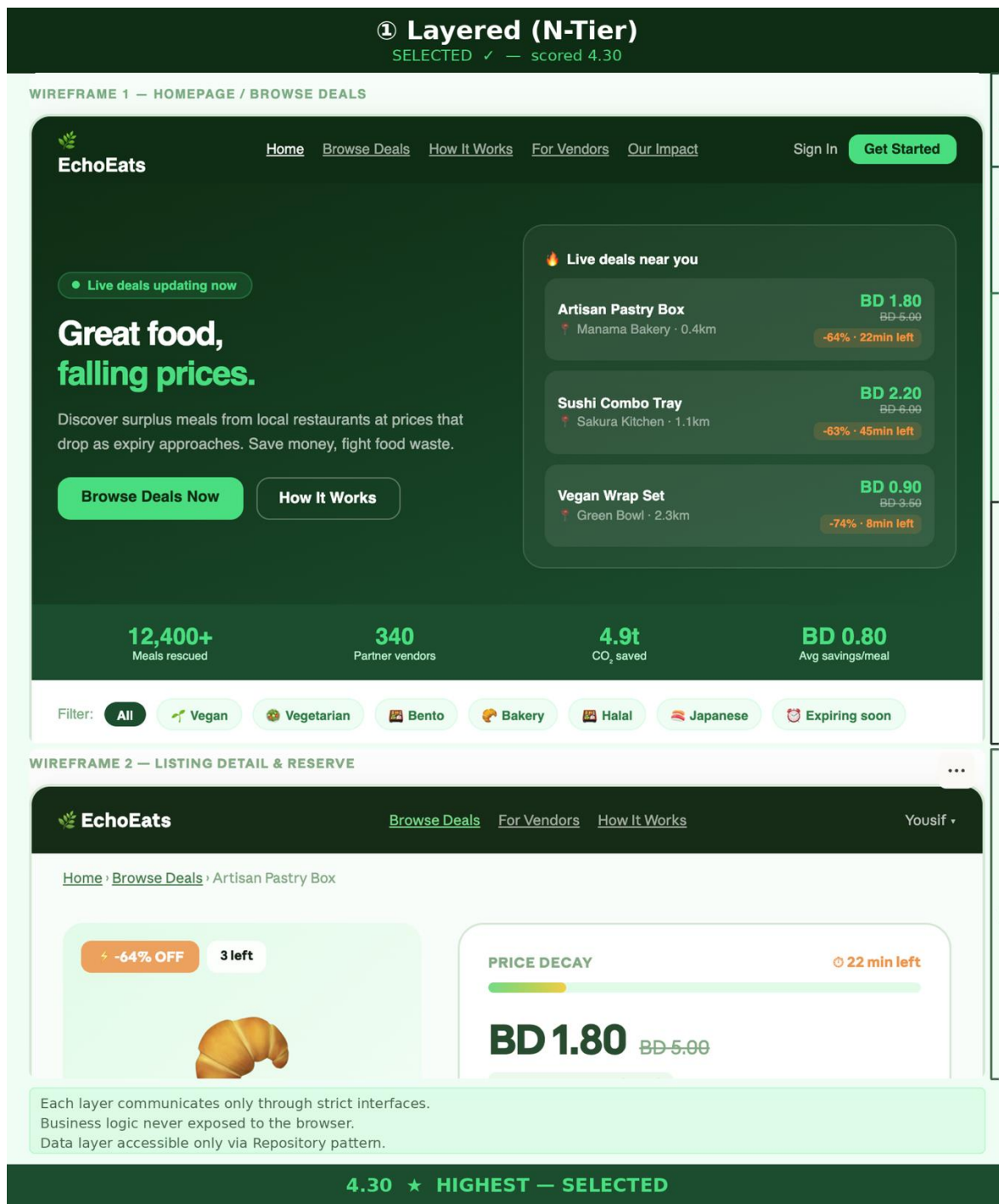
We assessed the fit of three architecture styles: real time dynamic pricing, two user types, geolocation, Stripe payments, and listing management scalability. The assessment was based on the boundaries of the team: five developers, four-week sprints, everyone using the GitHub codebase, and shipping working features every sprint.

Architecture Comparison

Six implicated criteria were assessed for three architectures. The N-Tier architecture organises the system into multiple horizontal layers called Presentation Layer, API Gateway Layer, Business Logic Layer, Data Access Layer, External APIs Layer. Each layer only communicates with the layer above it or below it. This creates clear ownership boundaries and renders each layer independently testable. Microservices architecture involves breaking the system down into distinct services that can be deployed independently (for example a listing service, a pricing service and a payment service, etc) where each would run in its own process. The simplest type of topology is Client-Server. A browser client communicates directly with a monolithic backend server.

Layered scored highest for maintainability and team fit: each of the five team members can own one or two layers without overlapping responsibilities, and the architecture requires no specialised DevOps knowledge to operate during development. Scalability was paramount for microservices, but team fit was a major concern. It was felt that the workload would consume the bulk of the sprint capacity on infrastructure work, not feature delivery, due to deploying and managing a service mesh, container orchestration, and distributed tracing. Client-Server is quick

to prototype, but at risk of tight coupling and does not send real time pricing updates without middleware.



② Microservices

Considered $\times$ — scored 3.35 — too complex for Sprint 2

API Gateway / Service Mesh

JWT auth · rate limiting · SSL · routing

Listing Service /api/listings

12,400+ Meals rescued 340 Partner vendors 4.9t CO₂ saved BD 0.80 Avg savings/meal

Filter: All Vegan Vegetarian Bento Bakery Halal Japanese Expiring soon

Deals ending soon near you

Artisan Pastry Box
Manama Bakery · 0.4km
0 22 min remaining

Vegan Bakery
BD 1.80
BD 5.00

Reserve →

Sushi Combo Tray
Sakura Kitchen · 1.1km
0 45 min remaining

Japanese Halal
BD 2.20
BD 6.00

Reserve →

Vegan Wrap Set
Green Bowl · 2.3km
0 8 min remaining

Vegan Gluten-free
BD 0.90
BD 3.50

Reserve →

Pricing Service /api/pricing

PRICE DECAY 0 22 min left

BD 1.80 BD 5.00

You save BD 3.20 (64%)

Quantity: - 1 + 3 remaining

Reserve Now · BD 1.80

Vendor Service /api/vendor

REFRAME 3 — VENDOR DASHBOARD

EchoEats Vendor Portal + Add Listing Manama Bakery

OVERVIEW

- Dashboard
- My Listings
- Reservations

ANALYTICS

- Revenue
- Pricing Curves
- Impact Report

ACCOUNT

- Settings

Welcome back, Manama Bakery

REVENUE (SPRINT)
BD 248.40
↑ +18% vs last sprint

MEALS SOLD
134
↑ +24 this week

AVG SELL PRICE
BD 1.85
Base: BD 5.00

CO₂ SAVED
42kg
↑ +8kg this week

Revenue by day (this week)

Mon	Tue	Wed	Thu	Fri	Sat	Sun
100	120	150	180	160	140	110

Active pricing curves

Your listings use 3 decay strategies. Linear performs best for pastries; Exponential drives urgency on last-hour items.

Reservation Svc /api/reservations

REFRAME 4 — RESERVATION CONFIRMATION & QR CODE

EchoEats Reservation confirmed

Reservation Confirmed!
Your QR code is ready · Show it at pickup

Your Pickup QR Code

Reservation Details

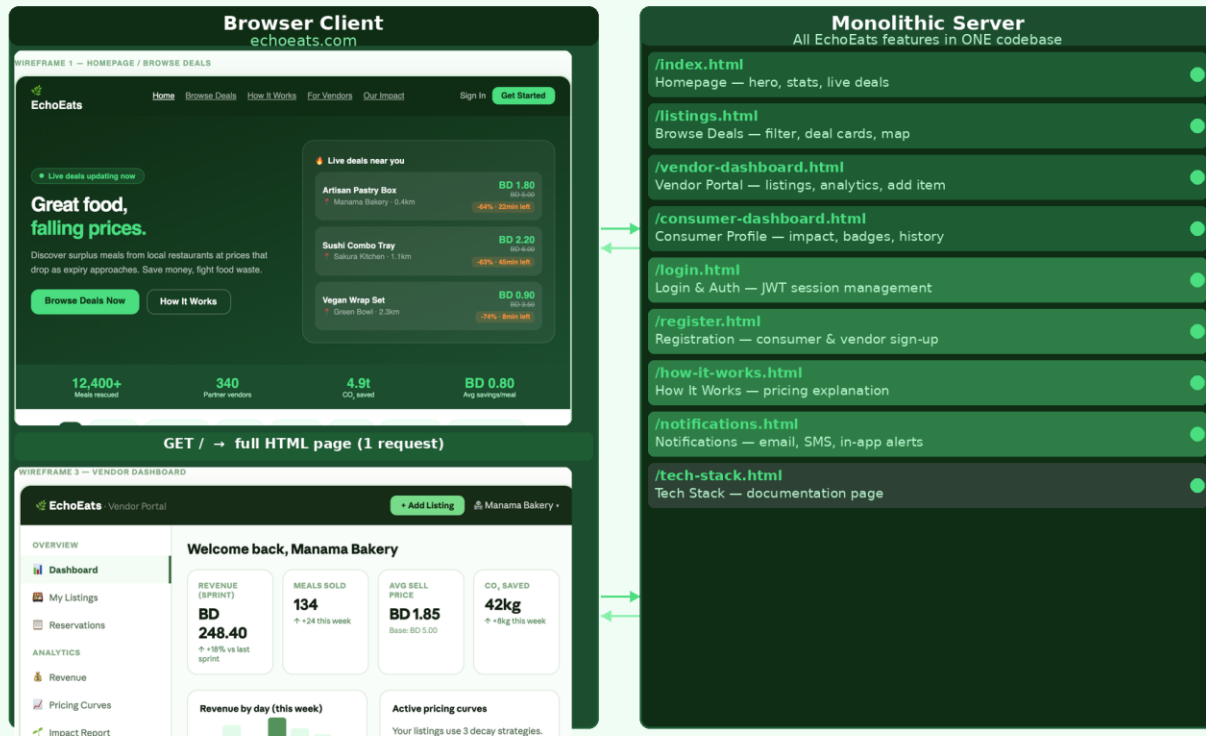
Item	Artisan Pastry Box
Vendor	Manama Bakery

Why NOT for EchoEats (Sprint 2):

- Service mesh + container orchestration = 2+ sprint weeks of DevOps
- 5-person team lacks DevOps specialisation for distributed CI/CD
- Correct evolution path — adopt at Sprint 4+ at production scale

③ Client-Server

Considered **X** — scored 3.30 — too limited for EchoEats



Why this fails for EchoEats:

- × No WebSocket → live price decay updates are impossible
- × Browse Deals traffic spike slows down Login and Vendor Dashboard too
- × Cannot deploy or scale any single feature independently
- × Pricing logic tightly coupled to HTML — impossible to unit-test cleanly
- × Single point of failure — one unhandled error crashes all 9 pages

3.30

Criterion	Layered (N-Tier)	Microservices	Client-Server
Scalability	Moderate — vertical scaling	High — services scale independently	Low — server bottleneck
Security	Good — single entry point via API Gateway	Moderate — more attack surfaces	Good — clear trust boundary
Maintainability	High — clear layer separation	Moderate — complex orchestration	Moderate — tight coupling risk
Team Fit	Excellent — 5-person, clear ownership per layer	Poor — DevOps expertise needed	Good — but limited feature isolation
Real-time Pricing	Possible — WebSocket added to API tier	Excellent — dedicated pricing microservice	Difficult without middleware
Delivery Speed	Fast — minimal infra overhead	Slow — service mesh + CI/CD pipelines	Fast — simple but rigid

Decision Matrix — Weighted Scoring

A relative weight was assigned to each criterion to demonstrate its relevance to EchoEats' stage. The weighted 25% for Scalability, as Platform must handle variable listing and reservation volumes but not at the magnitude requiring microservices in the semester. Maintaining the maintainability and fit of the team are each 20% because the team consists of five developers and works in shorter sprints. Given that it handles payments and user geolocation, security is weighted 20%. Delivery speed accounts for 15% because every sprint must include a shippable feature.



Criterion	Weight	Layered Score (×W)	Microservices (×W)	Client-Server (×W)
Scalability	25%	3 (0.75)	5 (1.25)	2 (0.50)
Security	20%	4 (0.80)	4 (0.80)	4 (0.80)
Maintainability	20%	5 (1.00)	3 (0.60)	3 (0.60)
Team Fit	20%	5 (1.00)	2 (0.40)	4 (0.80)
Delivery Speed	15%	5 (0.75)	2 (0.30)	4 (0.60)
TOTAL WEIGHTED SCORE	100%	4.30 ✓ SELECTED	3.35	3.30

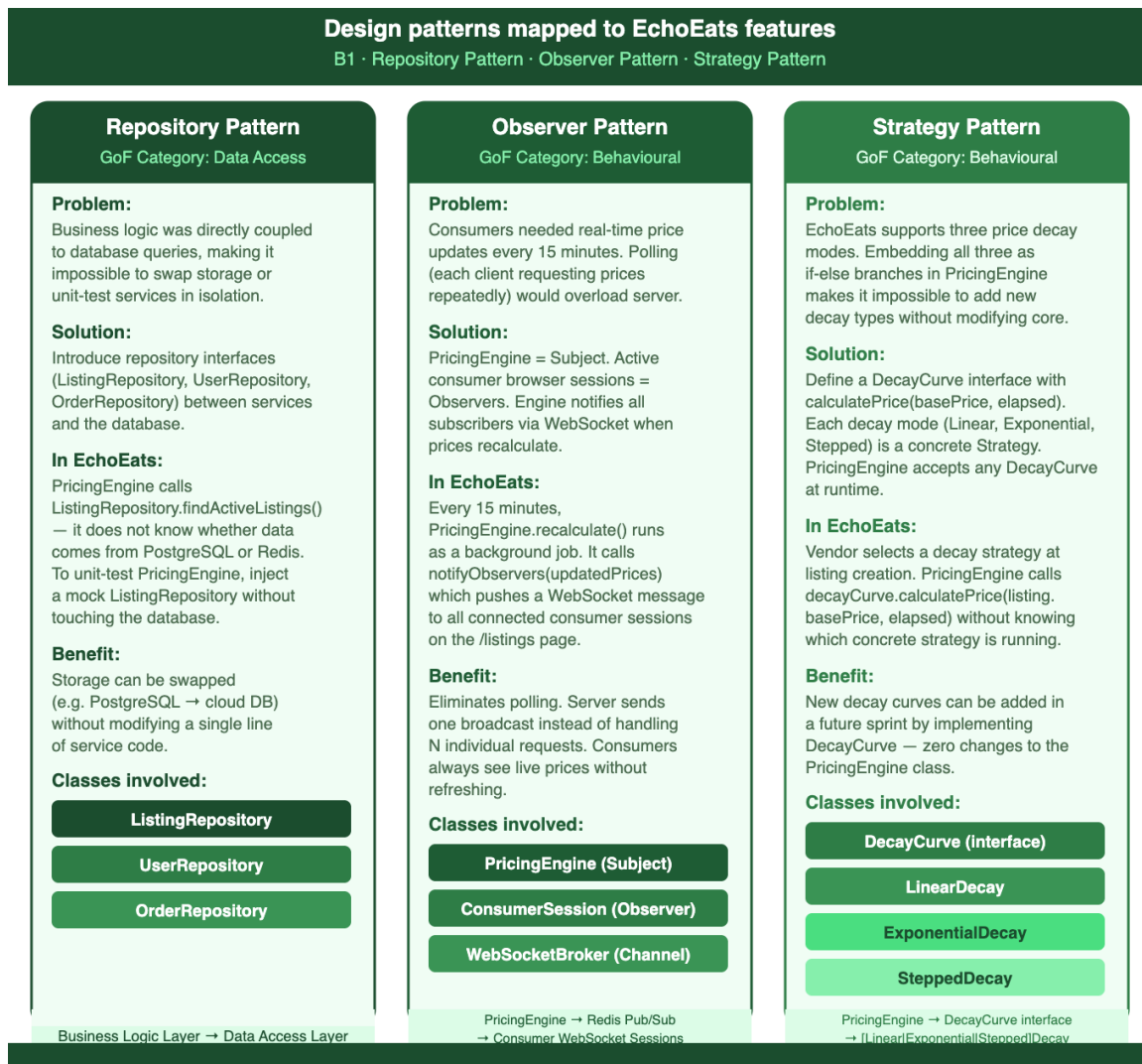
Selected Architecture: Layered (N-Tier) with MVC Application Layer

The layered architecture scored 4.30 and best fits EchoEats at this point. A five-member prototype squad wouldn't receive adequate value from microservices as it is likely to consume sprint capacity on infrastructure overhead (service mesh, container orchestration, distributed tracing, per-service CI/CD). Microservices continue to be the right direction for production-scale commercial food marketplace in Sprint 4+, but they are the wrong choice for a five-person team building a working prototype in a semester.

Each team member has a layer which he owns; Yousif and Ali own Presentation; Mohamed owns Business Logic; Hussain and Rashid own Data Access and Infrastructure. Ensures clear PR responsibility and averts bottlenecks. The five layers are Presentation, API Gateway, Business Logic, Data Access (PostgreSQL + Redis) and External APIs (Stripe, Google Maps, Push Notifications).

Design Patterns Mapped to EchoEats Features

The development of EchoEats led to the identification of three distinct design patterns such as Repository, Observer, and Strategy Patterns. Each was mapped to specific feature with each pattern was selected as to solve a concrete problem and not a theoretical exercise.



The Data Access layer consists of three repositories: ListingRepository, UserRepository, and OrderRepository. It separates the business logic layer from the implementation details of the database. The pricing engine and reservation service will call the repository methods without having to worry if the data is stored in PostgreSQL or if it is cached in Redis. It also means you can replace or upgrade the storage technology unA relative weight was assigned to each criterion to demonstrate its relevance to EchoEats' stage. The weighted 25% for Scalability, as Platform must handle variable listing and reservation volumes but not at the magnitude requiring microservices in the semester. Maintaining the maintainability and fit of the team are each 20% because the team consists of five developers and works in shorter sprints. Given that it handles payments and user geolocation, security is weighted 20%. Delivery speed accounts for 15% because every sprint must include a shippable feature.der the service without altering any service code.

The dynamic pricing engine uses the observer pattern to distribute live updates. Every 15 minutes, the PricingEngine recalculates decay curves for all active listings and notifies all subscribed consumer browser sessions via WebSocket. Those who are currently browsing the listings page are the observers. By drastically reducing the amount of polling consumers need to do, the load on the server is also greatly decreased. Furthermore, every active consumer always sees the latest price and does not need to refresh the page.

The Strategy pattern is used when choosing the decay curve at the time of listing creation. When creating the listing, the vendor picks one out of three decay strategies – linear, exponential and stepped. The PricingEngine takes a DecayCurve interface to execute the strategy of choice. As a result, the conditional branching in the pricing engine is eliminated, and in a future sprint, it will become easy to add another type of decay curve without changing existing engine code.

A.2b — Detailed Design Document (B2)

UML Class Diagram for the Core Domain Model

The domain model of EchoEats consists of 10 classes organized in 3 layers (User hierarchy, core transactional entities, and pricing strategy hierarchy). This diagram uses five relationship types, namely inheritance, composition, association, dependency, and realization. These were chosen to show how objects own, use, or depend on each other at runtime.

Hierarchy of users (inheritance).

The user is declared abstract because no object should be created as a user in general every account is either Consumer or Vendor. The user has share the attributes of both roles: `userId` (UUID primary key), `email`, `name`, `passwordHash`, `phone` and `createdAt`. The two subclasses can depend on the authentication contract (JWT Token) which is received as a response for `login()`. The Consumer class acts as an extension for the User class, and it includes `dietaryPrefs` (a String array used to filter listings), `savedListings` (UUIDs of wishlisted items), and `walletBalance` to its state. The consumer has a separate behavioural interface for the operations it offers. For example, `searchFood()`, `makeReservation()`, `pay()`, and `cancelReservation()` are operations that should only ever belong to a consumer. Further, these operations should never be accessible from a Vendor reference. A vendor provides the user with `businessName`; `verified` flag (set by way of a manual admin review before vendor goes live); `rating`; `revenueDashboard statsObject`. The complete vendor flow is signified by its functions `addListing()`, `removeListing()`, `updateListing()`, and `viewDashboard()`. By putting them in two separate subclasses instead of a single User class

with a role flag, we're forcing clients to base level interfaces only and not letting them use base class functionality to violate the Interface Segregation Principle. The API Gateway can type check against Consumer or Vendor at the service boundary. By placing the subclass at the service boundary, we can ensure to never expose vendor operations to a consumer session, and vice versa.

ImpactProfile and NotificationPreference (Composition). ImpactProfile has a one to one composition relation with consumer. We chose composition over association because we felt that an ImpactProfile has no real meaning outside the Consumer that it belongs to. In other words, if a consumer account is deleted, then the profile must be deleted too. ImpactProfile saves foodSaved (Float), co2Saved (Float in kg), streak (Integer), badges (String array). The gamification layer is supported by its methods addBadge() and getStats(). Consumer has a one to one composition with NotificationPreference PushEnabled, emailEnabled, quietFrom Time, alertRadius Float in km and dietaryFilters String array. To avoid errors on the first login, this class is created transactionally with the consumer account. It directly corresponds with the Notification Preferences page in the frontend.

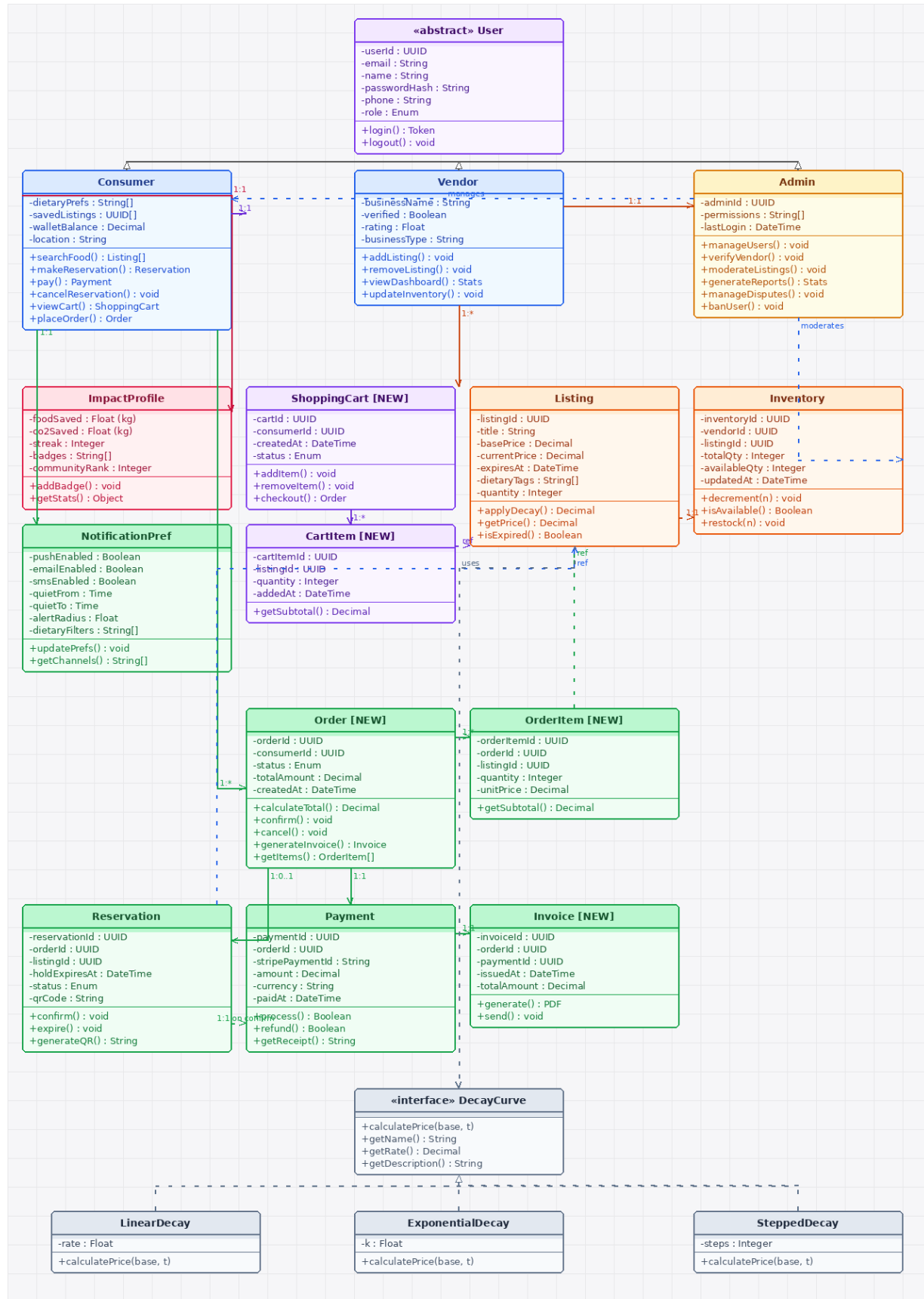
Listing of vendors and Activities (Association with Vendor, Dependency on DecayCurve). A vendor might have zero listings (as in newly created account) or many. Any vendor may also have a listing count of zero to unlimited, but each listing can only be part of a single vendor. A Listing should be linked to only one Vendor. The listing is the most attribute-rich class in the diagram because it carries all the pricing state the engine needs: listingId (UUID), title, basePrice, currentPrice, minPrice (the vendor-set floor), expiresAt (DateTime), and dietaryTags (String array). The use of applyDecay() methods is similar to methods used in [DecayCurve.it](#) is used to recalculate currentPrice. The cron tick that applies every 15 minutes runs applyDecay(). Furthermore, getPrice() returns current price for display. Finally, isExpired() returns Boolean used by auto-archiving job. The presence of a dashed dependency arrow from Listing to DecayCurve indicates that Listing is reliant on the aforementioned interface. However, Listing does not own DecayCurve or contain it as a subordinate. In effect, the strategy object is injected at listing creation time based on the curve that the vendor chooses.

Booking and Payment (Connection Chain).

The reservation links a consumer to a listing the consumer is interested in. It contains the main state of the 15 minute hold flow: the reservationId, listingId (FK to Listing), holdExpiresAt (DateTime, created as now + 15minutes), status (Enum: pending / confirmed / expired / cancelled), qrCode (String, which will be generated on payment confirmation). The three state transitions in the reservation lifecycle are confirmed using the expire() and generateQR() methods. There is a 1 to 1 association between Payment and Reservation : every confirmed reservation must have one payment record and a payment record that does not have a reservation record has no meaning. The payment records include a stripePaymentId, amount, currency, and paidAt. The Stripe SDK calls are wrapped by its process() and refund() methods. The constraints of multiplicity force the business rule that once a Reservation is created with status pending, that listing cannot be double-booked. Thus this listing is locked until the holdExpiresAt or payment confirmation.

Hierarchy of DecayCurve Strategy (Interface + Realization).

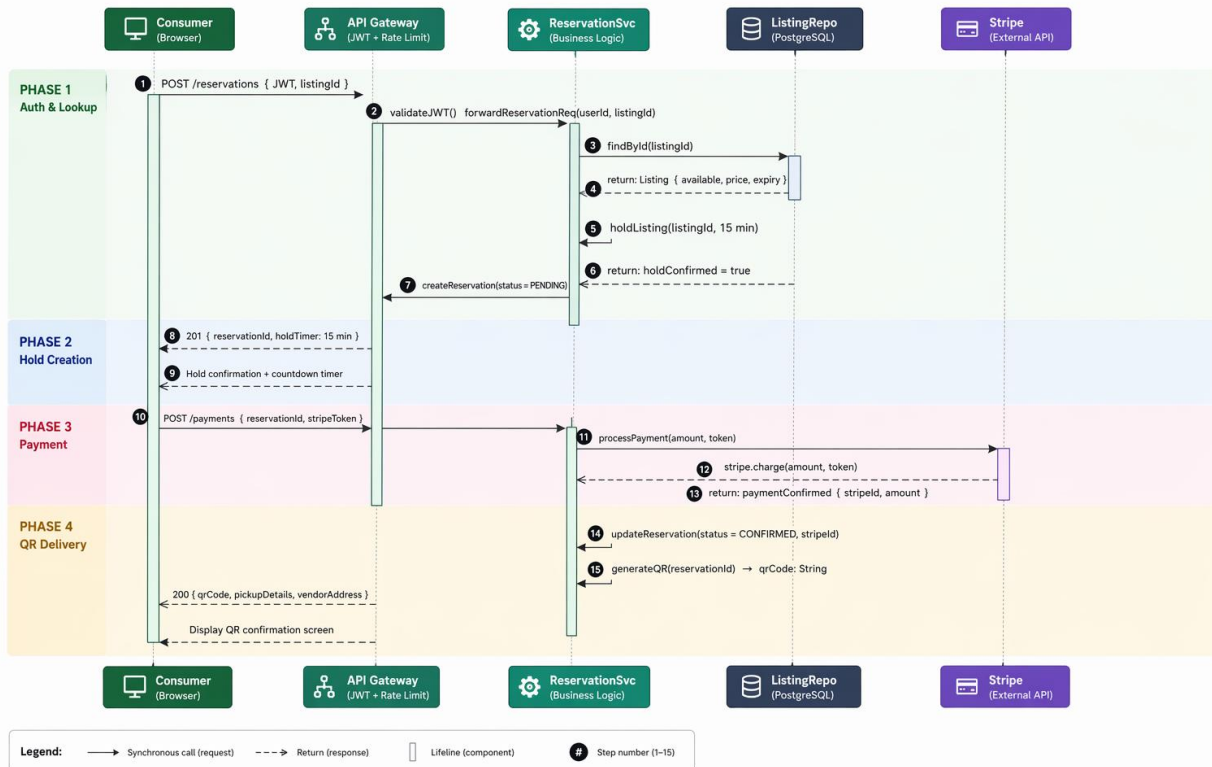
DecayCurve refers to an interface with the following three methods: 1) calculatePrice(base, t): Decimal (the main computation) 2) getName(): String (this is what shows in the vendor dashboard as the name of the active curve) 3) getRate(): Decimal (analytics). Three concrete classes provide this interface. The LinearDecay class contains a single rate attribute which applies a constant percentage reduction per tick, thus making it more predictable and easier for the consumers to plan around. The ExponentialDecay takes a constant k and then applies the formula $e^{k(t-T)}$. This makes the price hover around basePrice for most of the listing period. However, in the last hour, the price will drop sharply leading to a sense of urgency to purchase. SteppedDecay takes in steps (Integer) and thresholds (Float array) and drops the price at preset milestones for example, to 90% of basePrice, then to 70%, then to 50%, then to 30%. By applying the Strategy pattern here, we ensure that the PricingEngine's applyDecay() method has no conditional branching at all. It simply calls calculatePrice() on whatever DecayCurve instance was assigned to the listing. Adding a fourth curve type on a future sprint (for example) will need us to write a new class that implements the interface. There will be no change to Listing nor to PricingEngine.



Sequence Diagram — Reserve a Listing (Core Flow)

EchoEats — Sequence Diagram: Reserve a Listing (Core Flow)

Figure 4 · 15 Steps · Sprint 2



The complete Reserve a Listing flow gets modelled in the sequence diagram, which follows the numbered steps (1-15). It contains four phases: Auth & Lookup (steps 1-4), Hold Creation (steps 5-8), Payment (steps 9-12), and QR Delivery (steps 13-15). The lifelines participating: Consumer (browser), API Gateway; ReservationService; ListingRepository; Stripe. We selected this flow as the primary diagramming focus as it is the highest value, highest risk in the system. The flow includes an authentication, a database write that is susceptible to a race condition, an external payment call and a state machine transition, all of which must succeed in the right order for a reservation to succeed.

Phase 1 — Auth & Lookup (Steps 1–4).

The consumer sends a POST /reservations request carrying a JWT in the Authorization header and the target listingId in the request body. At Step 2, the API Gateway extracts and validates the JWT before forwarding anything downstream. This is the security gate: an expired, malformed, or missing token causes the gateway to return HTTP 401 immediately, and no message is ever sent to ReservationService. This step was absent in the first diagram draft and was caught during peer review (PR #14) — its absence would have implied that unauthenticated requests could reach the service layer, directly contradicting the security contract defined in the Component

Diagram. At Step 3, the validated request is forwarded to `ReservationService`. At Step 4, `ReservationService` calls `ListingRepository.findById(listingId)`. Two conditions are checked: the listing must exist and must not yet have passed its `expiresAt` timestamp. If either check fails, `ReservationService` returns HTTP 404 or 410 (Gone) and the flow terminates without creating any hold record.

Phase 2 — Hold Creation (Steps 5–8).

This is the most concurrency-sensitive phase in the system. At Step 5, `ReservationService` calls `ListingRepository.holdListing(listingId, consumerId)`, which executes a select for update query against the `Listing` row. This places a row-level lock on the record for the duration of the transaction. Any concurrent `holdListing()` call targeting the same listing blocks at this point until the first transaction commits or rolls back so this is the mechanism that makes double-booking structurally impossible regardless of concurrent traffic. At Step 6, a new `Reservation` record is inserted with status: pending and `holdExpiresAt` set to now +15 minutes. Step 7 commits the transaction, releasing the lock. Step 8 returns HTTP 201 to the consumer with the `reservationId`, the locked price, and the `holdExpiresAt` timestamp. The browser renders a countdown timer at this point, giving the consumer a clear visual signal of how long they have to complete payment before the hold lapses and the item becomes available to others.

Phase 3 — Payment (Steps 9–12).

The consumer sends a request (POST) to the reservations endpoint with the JWT included in the authorization header, and the `listingId` included in the request body. At Step 2, the API Gateway obtains the JWT from the authorization header and validates the JWT. If the JWT is found to be invalid for any reason (expired, incorrect format, or is absent), the Security Gateway responds to the consumer with an HTTP 401 status code. No message is sent to the `ReservationService`. This step was identified as being absent in the initial draft of the diagram in a peer review (PR #14) of that draft. The omission of this security step will allow requests to reach the `ReservationService` without authorization, which is contrary to the security contract that is established between the components in the Component Diagram. The `ReservationService` receives a go-ahead signal at Step 3. At the 4th step, the `ListingRepository` performs its lookup of the listing using the `listingId` that was received from the request body, and confirms that the listing is active and that the hour for that listing has not passed its `expiresAt` timestamp. If either of these steps fail, the `ReservationService` will return an HTTP 404 or 410 (Gone) response code to the consumer, and the process will terminate without the creation of any record of that reservation in the system.

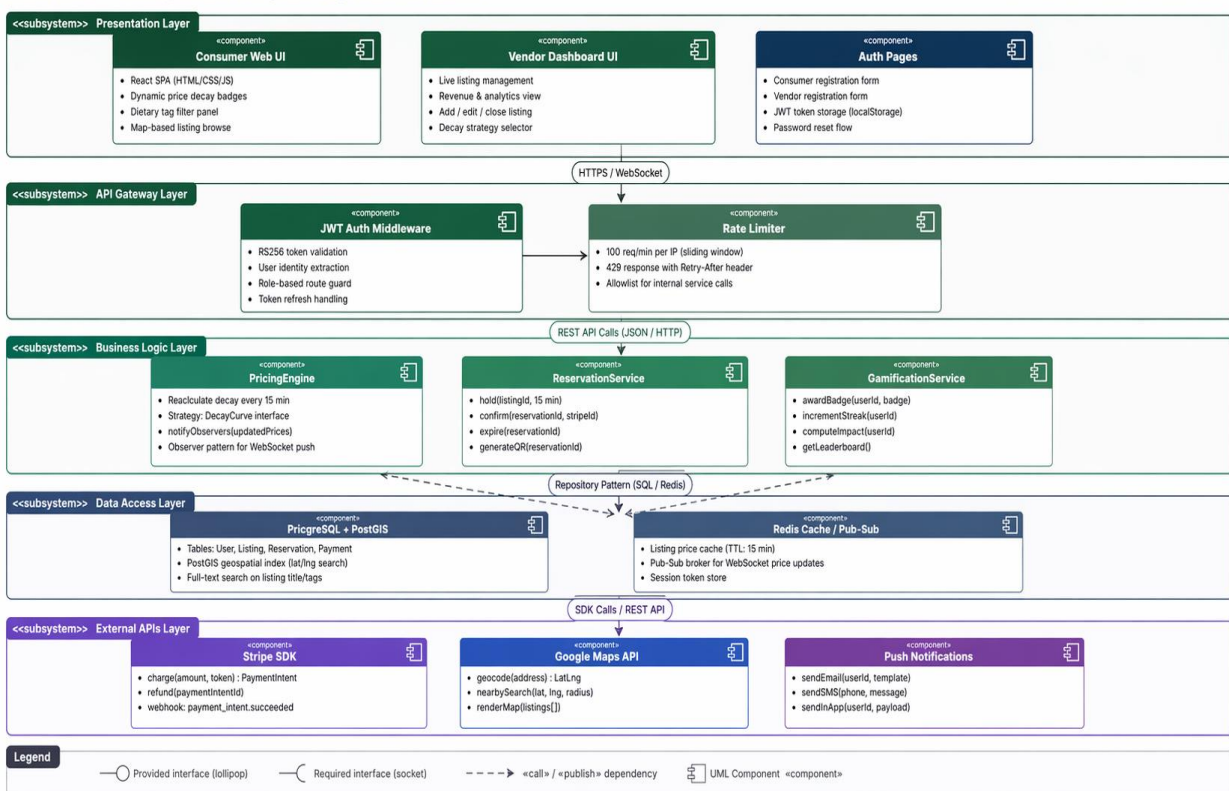
Phase 4 — QR Delivery (Steps 13–15).

The `generateQR()` called on the Reservation entity in Step 13 take the alpha numeric pickup code composed of the reservationId as a unique value. This pickup code is stored in the Reservation entity in Step 14. Step 15 Reserves Step 15 sends the reservation containing the generated QR code, vendor address, and new ImpactProfile statistics of the consumer back to the browser. The artefact that the consumer will require when collecting the food is this QR code - when scanned by the vendor, the latter will get access to the reservation without any details regarding payment on the particular order.

Component Diagram — System Modules

EchoEats — Component Diagram (System Modules)

Figure 5 · UML Component Diagram · Sprint 3



The Component Diagram structures EchoEats into 5 vertical layers, each having a certain responsibility area and a predetermined interface contracts, which involves the interactions between the neighbouring layers. Communication is not allowed between one layer with another layer that is not an adjacent. This does not allow any reverse-level dependency rule of the N-Tier architecture: classes on upper levels cannot depend at all on the classes on lower levels.

Layer 1 — Presentation.

The Consumer Web UI (listing detail, the reservation screen, the browse screen, and the listing management) and the Vendor Dashboard UI (listing management, analytics, the add listing form, and the registration flows and the login screen) each have three layers, with the Auth Pages as the bottom one. All the three components are delivered in the form of static HTML/CSS/JS files. They only communicate to the API Gateway layer using two protocols: HTTPS, to make simple request-response requests (login, fetch listings, post reservation), and WebSocket, to interpret the events on live price update that are being pushed by the PricingEngine. This layer also has a purpose in the existence of Consumer and Vendor UIs so that the consumer facing screens are not affected by deploying the vendor dashboard or updating or managing it based on access needs.

Layer 2 — API Gateway.

Positioned between the Presentation and Business Logic layers, the API Gateway has three critical functions to perform prior to any requests reaching the services. The API Gateway validates JWT tokens for each request (rejecting requests from unauthenticated clients with an HTTP 401 response code), applies rate limiting to reject requests from any IP address making 100 requests per minute (to prevent abuse of the service), and routes incoming requests to the appropriate service via REST calls. The API Gateway has no session data of its own. Furthermore, the WebSocket upgrade from HTTPS is also processed by the API Gateway, allowing the PricingEngine to push real-time events to the clients browsing the application. By centralising the control of the API Gateway instead of implementing the same functionality within each service, there are fewer opportunities for errors and the authentication is managed at a single point.

Layer 3 — Business Logic.

Three services operate within this layer. The PricingEngine forms the core of EchoEats and works in cron every 15 minutes to iterate through all active listings and call the `calculatePrice()` function on each of them based on the time passed since the listing's creation and saves the calculated price. It also publishes an event to Redis so that the WebSocket connection can broadcast the message to all connected clients. The ReservationService handles the hold functionality of the listings as described in the Sequence Diagram. Finally, the GamificationService is triggered upon each reservation confirmation and updates the ImpactProfile and badges of the consumer.

Layer 4 — Data Access.

The main data store along with users, listings, reservations, payments and impact statistics is PostgreSQL 16 (enhanced with PostGIS). Thanks to PostGIS, there's the nearby vendor search query, where the consumer's coordinates are passed to a ST_DWithin geo query that returns all listings in a specified radius. This is done in under 50 milliseconds. To prevent double booking at the database level, I will add row-level locking on the Listing table (SELECT FOR UPDATE). With every tick of our pricing engine, Redis sits alongside Postgres. It has two jobs – to cache the current price of every active listing. This means every consumer page load can read from memory at <1ms instead of reading from Postgres at ~5ms. Redis also has the role of being the pub/sub message broker. It carries our pricing engine update events to the websocket layer above. The bottom of the stack integrates three third-parties. Stripe does payment processing through PaymentIntents and webhooks.

Layer 5 — External APIs.

None of the card information accesses the EchoEats servers. As a result PCI compliance is easy. With Stripe Connect, the vendor payouts are taken care of through an automatic splitting of a platform fee of 15%. The balance amount gets credited to the vendor's linked account. The consumer browse view and the map search screen utilize Google Maps services for geocoding and map tile. Once a badge is alerted or a streak-alert-at-risk push notification is sent, alerts about 1 hour till expiry and 30 minutes till expiry are configured in the NotificationPreference class. Push Notification service with SendGrid (for email) and device notification provider for push are triggered by GamificationService.

High-Fidelity Wireframes — 4 Core Screens

Screen 1 features the consumer browse view. The app incorporates a map that pins vendors, a listing card that displays decay badges live prices of the item. And at the top is the filter bar for dietary preferences to show foods that are vegan, halal, gluten free and a few more. Screen 2 displays the details of the listing. Consumers have the ability to tap on an item to view the decay progress bar, which indicates how long the item has left before it expires along with the amount of that item being sold at this current live price along with a Reserve button. Screen 3 shows us the vendor dashboard. A trader is controlling everything here, so the live listings are live with their prices and expiry timers. At the top, you'll see a summary of my revenue and an Add Listing button that opens a form. It is the post-payment confirmation screen at point number 4. The QR code that will be visible is the one the consumer will be using at checkout. An updated impact counter has been added, allowing consumers to know how much food they are saving. Gamification functionality of the app is also benefited by this.

WIREFRAME 1 — HOMEPAGE / BROWSE DEALS



EchoEats

Home

Browse Deals

How It Works

For Vendors

Our Impact

Sign In

Get Started

• Live deals updating now

Great food,
falling prices.

Discover surplus meals from local restaurants at prices that drop as expiry approaches. Save money, fight food waste.

Browse Deals Now

How It Works

🔥 Live deals near you

Artisan Pastry Box

📍 Manama Bakery · 0.4km

BD 1.80

BD 5.00

-64% · 22min left

Sushi Combo Tray

📍 Sakura Kitchen · 1.1km

BD 2.20

BD 6.00

-63% · 45min left

Vegan Wrap Set

📍 Green Bowl · 2.3km

BD 0.90

BD 3.50

-74% · 8min left

12,400+

Meals rescued

340

Partner vendors

4.9t

CO₂ saved

BD 0.80

Avg savings/meal

Filter:

All

Vegan

Vegetarian

Bento

Bakery

Halal

Japanese

Expiring soon

🕒 Deals ending soon near you



-64%

Artisan Pastry Box

📍 Manama Bakery · 0.4km

⌚ 22 min remaining

Vegan

Bakery

BD 1.80

BD 5.00

Reserve →



-63%

Sushi Combo Tray

📍 Sakura Kitchen · 1.1km

⌚ 45 min remaining

Japanese

Halal

BD 2.20

BD 6.00

Reserve →



-74%

Vegan Wrap Set

📍 Green Bowl · 2.3km

⌚ 8 min remaining!

Vegan

Gluten-free

BD 0.90

BD 3.50

Reserve →

EchoEats Team

WIREFRAME 2 — LISTING DETAIL & RESERVE

EchoEats

[Browse Deals](#)[For Vendors](#)[How It Works](#)

Yousif ▾

Home › [Browse Deals](#) › Artisan Pastry Box

-64% OFF

3 left



Artisan Pastry Box

Manama Bakery

✓
Verified

★★★★★

4.9 (127 reviews)

Vegan

Bakery

Gluten-Free Option

0.4km away

ABOUT THIS LISTING

A beautiful selection of freshly baked croissants, pain au chocolat, and seasonal tarts. Made fresh this morning from locally sourced ingredients. Pick up before 8 PM tonight to rescue these pastries from waste.

PICKUP DETAILS

Manama Bakery, Block 321, Manama

Show QR code at the counter on arrival

Pickup window: 7:00 PM – 8:00 PM today

Listing expires in 22 minutes

Payment secured via Stripe

Instant QR code delivered after payment

PRICE DECAY

22 min left

BD 1.80

BD 5.00

You save BD 3.20 (64%)

Quantity:

-

1

+

3 remaining

Reserve Now · BD 1.80

15-min hold while you complete payment via Stripe

Your impact if you reserve:

~320g

Food saved

+15pts

Eco streak

0.4kg

CO₂ avoided

EchoEats Team

WIREFRAME 3 — VENDOR DASHBOARD

 EchoEats · Vendor Portal

+ Add Listing

 Manama Bakery ▾

OVERVIEW

Dashboard

My Listings

Reservations

ANALYTICS

Revenue

Pricing Curves

Impact Report

ACCOUNT

Settings

Welcome back, Manama Bakery

REVENUE (SPRINT)

BD 248.40

↑ +18% vs last sprint

MEALS SOLD

134

↑ +24 this week

AVG SELL PRICE

BD 1.85

Base: BD 5.00

CO₂ SAVED

42kg

↑ +8kg this week

Revenue by day (this week)

Mon

Tue

Wed

Thu

Fri

Sat

Sun

Active pricing curves

Your listings use 3 decay strategies. Linear performs best for pastries; Exponential drives urgency on last-hour items.

Linear (5 listings)

Exponential (2)

Stepped (1)

Active listings



Artisan Pastry Box

⌚ 22 min left · 3 remaining

BD 1.80

~~BD 5.00~~

● Live

Edit

Close



Sourdough Loaf (2-pack)

⌚ 1h 45min left · 8 remaining

BD 2.50

~~BD 6.50~~

● Live

Edit

Close



Cheesecake Slice (×4)

Expired 15 min ago

BD 0.80

~~BD 4.00~~

× Expired

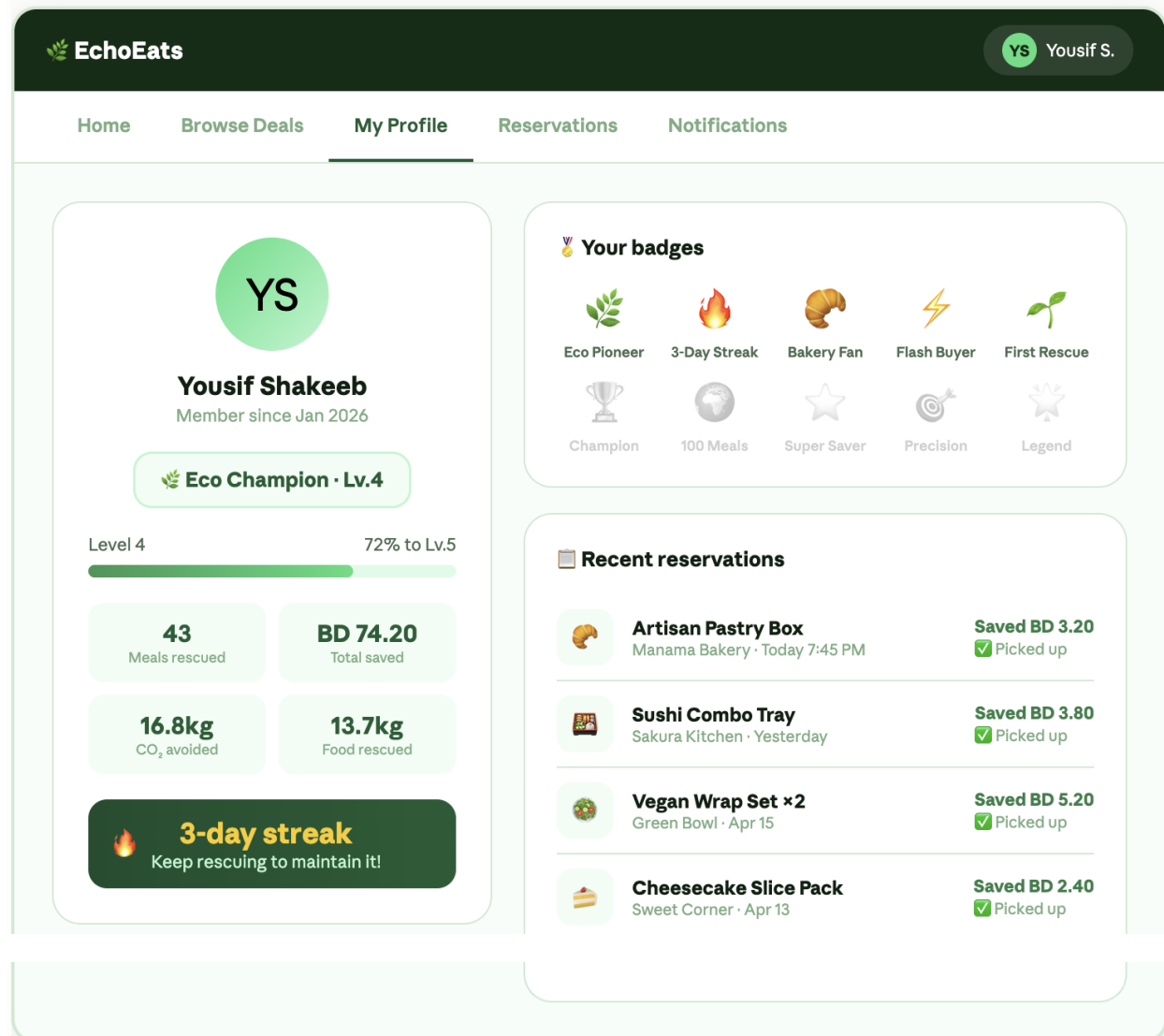
Re-list

EchoEats Team

...



WIREFRAME 5 — CONSUMER IMPACT PROFILE & GAMIFICATION



Ethical Design Statement

EchoEats operates within a domain that involves the processing of user location data, payment processing, and food safety. Ethical concerns were considered when designing the architecture during Sprint 1 through Sprint 4, with ethics being considered not as a list of requirements to be adhered to but rather as design constraints influencing actual architectural decisions.

Privacy

Your location is obtained only while browsing restaurant listings and having reservations active. The latitude and longitude values are stored for a period of 48 hours before being deleted. The vendor receives information about the general geographical area where the consumer resides but does not receive their precise latitude and longitude values.

Fairness

All listings under the same category use the same decay function. A consumer does not see a higher price compared to other consumers of the same listing due to his/her profile or past purchases. The demand multiplier that might result in a slower decay of the listing price due to increased views is restricted to a maximum of 1.0. This implies that it would slow down rather than increase the price. Our dietary tag filtering process does not exclude any ethnicity-based or religion-based food labels from being filtered out.

Safety

A listing cannot be created using an expiry time in the past. Listings will expire automatically depending on the timestamp. Tags for food allergens are mandatory for all high-risk food categories. Manual vetting is mandatory for all vendor profiles within 24 hours after creation of the account. Non-vetted food suppliers cannot create listings.

Ethical Trade-off: Convenience vs. Privacy

The biggest ethical dilemma faced by the team pertained to giving permission for ongoing location tracking in the background for discovering deals through push notifications. If location permissions were granted in the background, then the platform could notify the user when they were within the vicinity of any active deal. Post deliberation during Stand-up #2, there came about a consensus among the team members whereby it was decided to forego this feature in favor of our own stringent privacy policy. In other words, location tracking would be done on a just in time basis only once the consumer accessed your application to browse the available deals.

PART 2 — INDIVIDUAL REPORT | Yousif Shakeeb · Developer · Sprint 2

Name	Yousif Shakeeb	Role	Developer
Sprint	Sprint 2	Student ID	F2400136

Part 2 — Individual Contributions

My Role as Developer

However, my experience in Sprint 2 was completely different from that of Sprint 1. Being the scrum master in Sprint 1, my duty was to do the stand up and update the Kanban board. But this time around, I left everything on Rashid Bomtia and became a developer myself. It was not difficult for me to become a developer since he had already started working technically from Sprint 1 onwards. For the rest of the group members, it had been an easy task since we were accustomed to the stand up process and Kanban as well since Sprint 1 itself.

The UML Class Diagram that I did, I managed to complete the design for all the components of the domain model. The Presentation Layer was created in the Component Diagram. Also, I reviewed the Sequence Diagram by Mohamed where a crucial JWT validation was missed before merging.

2. What I Built

Repository: <https://github.com/YMShakeeb/echoeats-web>

Below is a description of each individual contribution that relates to the respective commit/pull requests in GitHub.

UML Class Diagram – Full Domain Model

I developed and wrote the whole UML Class Diagram myself, including the creation of each class, relationship, multiplicity, and method signature for all nine classes. The most important choice was the User hierarchy. Some members recommended a unified User class and a role attribute in Stand-up #3. However, Consumer and Vendor not only have distinct attributes; their behaviour is entirely different. A Consumer invokes `searchFood()`, `makeReservation()`, and `pay()` methods. A Vendor invokes `addListing()`, `removeListing()`, and `viewDashboard()` methods. One class would give every instance of users access to all methods regardless of their roles, violating Interface Segregation and requiring the API Gateway to perform run-time checks that are unnecessary. A subclass for each kind of User solves both issues. Regarding the reservation process, Reservation is the join entity between Consumer and Listing, containing `holdExpiresAt`, `status`, and `qrCode` fields. The Payment entity relates to the Reservation entity with a 1-to-1 relationship. In terms of pricing, I applied the Strategy design pattern, where `DecayCurve` is an interface implemented by `LinearDecay`, `ExponentialDecay`, and `SteppedDecay`. The final piece was `ImpactProfile`, which was added as a 1 to 1 composition of Consumer after evaluating the user stories of gamification from Sprint 1. Composition guarantees that it is transactionally created along with the account

ImpactProfile Composition

As I looked back at the gamification user stories in Sprint 1, I did not find anything that depicted where the data of badge and streaks would be stored. Hence, to address this problem, I included them in a 1 to 1 composition on the Consumer so that they are created along with the creation of the account rather than being lazily instantiated on first use. What is important about using a composition relationship for this problem is that in case the creation of an account is only partial, the consumer created will not have an impact profile, which would create silent issues in Sprint 3.

Owned Presentation Layer in Component Diagram

The components of the Presentation layer I have identified are Consumer Web UI, Vendor Dashboard UI, and Auth Pages, while the contracts for each component were defined in terms of HTTPS and WebSocket. It was used as reference by Mohamed for designing the API Gateway.

Peer Review: Sequence Diagram (PR #14)

While analyzing Mohamed's Sequence Diagram PR, I observed that there was no validation of JWT before the consumer issued the POST /reservations request. This step is necessary to ensure that the unauthenticated requests cannot be forwarded to the service layer, and this was contrary to the security design proposed in the Component Diagram. I pointed out this issue in my PR comments. Mohamed rectified this mistake before merging his changes.

Updated Consumer Wireframes — Browse and Listing Detail Screens

Following the feedback gathered from Sprint 1 evaluation and discussion in Stand-up #4, I redesigned Screen 1 and 2 with the inclusion of the decay progress bar and dietary tag filter respectively. The decay progress bar may seem like a small feature, but it serves a vital purpose by visually indicating the urgency of product expiration without the consumer having to manually compute the time remaining.

The key design decision that I made was to use inheritance instead of role flags, which I justified in Section 2. The other components under my sole responsibility include the reservation workflow, strategy pattern for decay curves, and ImpactProfile composition.

The addition of the ImpactProfile composition needs its own special attention. In the class diagram meeting, the gamification component (user stories US10-US12 in Sprint 1) had no corresponding classes defined. ImpactProfile, in the form of a 1 to 1 composition on the User class, ensures that the gamification profile is created transactionally when the user account is created. If this design wasn't implemented, gamification data could have been created lazily, leading to users using the application without having a gamification profile.

4. The Ethical Decision I Drove

My ethical contribution within Sprint 2 had to do with the concept of fairness when implementing the demand multiplier. In Week 5 during Stand-up #2, the issue was about finding the best way of going about the implementation of the demand multiplier, which determines the rate at which the price will decay depending on the level of engagement on a certain listing on the platform. Naturally, the idea behind the implementation of such a function is to consider aspects such as consumers' purchase behaviour and their level of engagement within the platform. This implies that a consumer with high engagement will enjoy slow price decay compared to a consumer using the service for the first time on the same item.

This approach to the issue of demand multiplier was problematic since it results into two-tier pricing. EchoEats serves the purpose of providing cheap and affordable surplus food products, but the people who need it the most are likely to be new consumers rather than those who use the platform regularly.

Two constraints were suggested by me: firstly, only aggregate views per listing will be taken into account in calculating the demand multiplier within the ongoing session window, and secondly, the value of the multiplier cannot exceed 1.0, therefore, its contribution to the calculation will only slow down the pace of price decay, but prices will never be set above the vendor's base price. In other words, the buyer will have to pay an equal price for the same item regardless of his/her time choice.

Both constraints were accepted by the team in Stand-up #2 and included in the Ethical Design Statement in terms of Fairness. They were hard-coded in the PricingEngine class as constraints. The cost of implementing such constraints is lower accuracy of the pricing model due to taking aggregate views rather than analyzing user behavior. Still, fairness is more important for the team in this case.

5. Conclusion

Things I did well: Transitioning from the Scrum Master to Developer role was easy because Rashid had been a very active developer in Sprint 1 and didn't require much induction into the role of Scrum Master. Parking-lot technique which I introduced to the team in Sprint 1 also proved to be effective while working under Rashid, as all the eight daily stand-ups managed to stay within the 15-minute time-box. Peer reviews performed on pull requests were the best collaboration result of this sprint: reviewing each pull request by at least two other team members helped catch an error in Mohamed's Sequence Diagram JWT issue. What I would do differently: I delayed the wireframe refinement until Week 8. However, the wireframes appear to be referenced in the Sequence Diagram and Component Diagram; hence, my work in Week 8 required revisions in those diagrams. In Sprint 3, I will finalize the work on wireframes no later than Week 6 so that everything else that references wireframes would also be able to refer to a finalized version.

Finally, Sprint 2 was an indication of what can be achieved when the process is running smoothly and each member knows their lane. There was a seamless transition for the role of Scrum Master to Rashid; there were no problems regarding context switching and adjustment. The format of conducting stand-ups in the parking lot ensured that each of the eight stand-up meetings conducted during Sprint 2 did not exceed 15 minutes and also identified potential blockers ahead of time, allowing for the issues to be addressed. It was evident that the rule regarding two reviewers per PR was beneficial, as it helped identify the problem regarding the missing JWT authentication step on the Sequence Diagram that Mohamed provided before merging to main.

Submitted for Academic Purposes · SWEN 360 / CMPE 460 · Spring 2026